



COM3110 Text Processing Assignment: Sentiment Analysis of Movie Reviews

Vivek Vijay Choradia

Date: May 25, 2024

1 Implementation

- **Prior Calculation:** In the *MultinomialNaiveBayes* class's *_calculate_class_probabilities* method, prior probabilities for each class are computed using Laplace Smoothing. The resultant class probabilities, accounting for even unseen features, are stored in the *class_probabilities* dictionary, crucial for the model's prediction phase. This method effectively mitigates the issue of zero probability in the presence of novel features in the test set.
- **Likelihood Calculation:** In the *MultinomialNaiveBayes* class, likelihood calculation is executed through the *_calculate_feature_probabilities* method. This process involves iterating over each class label and its corresponding feature counts, as stored in the *feature_counts_per_class* dictionary. For each feature within a class, the likelihood is computed using the formula $P(\text{feature}|\text{class}) = \frac{\text{Count of feature in class} + \alpha}{\text{Total words in class} + \alpha \times \text{Vocabulary length}}$. This approach, which includes Laplace smoothing via the parameter alpha, ensures non-zero probabilities, crucial for handling features not encountered in the training set. The resulting probabilities are then stored in the *feature_probabilities* dictionary for each class.
- **Feature Extraction:** Multiple feature extraction methods were employed for the tasks which could be found under the features directory. The most efficient ones were VADER in *vader.py* and VADER + TFIDF in *combined.py*. VADER (Valence Aware Dictionary and sEntiment Reasoner) sentiment analysis tool from the Natural Language ToolKit (NLTK) library is designed to assign a sentiment score to each phrase in the dataset, with the scoring system ranging from -1 to 1 (signifying two ends of sentiment scale). This scoring mechanism is facilitated by the *SentimentIntensityAnalyzer* from the NLTK library, which adeptly calculates polarity scores for each sentence. In the *combined.py* approach, the VADER sentiment analysis tool is integrated with the TFIDF model from *scikit-learn*, combining broad sentiment scoring with context-sensitive word importance for more nuanced sentiment analysis. This method enhances accuracy by balancing general sentiment with the specific weight of terms in the dataset.
- **Macro-F1 Score:** The macro-F1 score is calculated through a two-step process in file *macro_f1_score.py*. For each unique label, it computes the F1 score individually using $F1\text{-score}_i = \frac{2 \times \text{Precision}_i \times \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i} = \frac{2 \times TP_i}{2 \times TP_i + FP_i + FN_i}$ implemented in *calculate_f1_score()*, which determines true positives, false positives, and false negatives for that specific label, and subsequently calculates precision and recall. The macro-F1 score is then derived as the arithmetic mean of these individual F1 scores. The overall macro-F1 score, returned, serves as a comprehensive metric to assess the performance of the sentiment analysis model implemented.
- **Pre-processing:** The model employs three pre-processing settings which are aimed to reduce noise and improve interpretative precision:
 - No Preprocessing (NP)
 - Preprocessing 1 (P1) = lowercasing + stopwords removal
 - Preprocessing 2 (P2) = lowercasing + handling contractions + tokenisation + stopwords removal + stemming + lemmatization
- **Code Efficiency:** Enhanced through the use of efficient data structures like *defaultdict* and *Counter*, vectorised operations in *numpy*, and data manipulation with *pandas*. Modular and object-oriented programming practices further contribute to the code's performance.

2 Feature Selection

In sentiment analysis, the selection of features is paramount to the performance of the classifier. Our empirical investigation into various feature sets has led to two primary feature selections: VADER alone and VADER combined with TFIDF. This selection process has been guided by exhaustive feature exploration, aiming to optimize Macro F1 Scores across both 5-value and 3-value sentiment classification tasks.

The hypothesis underpinning the feature selection is that sentiment in reviews is multi-faceted: it is directly expressed through polarity words, for which VADER is well-suited, and it is also contextually enriched by the relative importance of terms, which TFIDF captures.

VADER for 3-Value Sentiment Analysis:

VADER, a lexicon-based approach, demonstrated exceptional performance on the 3-value scale, particularly with P1 setting, as reflected by a Macro F1 Score of 0.416. This aligns with the lexicon's design, which effectively captures the polarity of sentiments in a more generalized fashion, suitable for the tripartite classification of sentiments. [1]

VADER + TFIDF for 5-Value Sentiment Analysis:

In contrast, the combination of VADER and TFIDF excelled in the 5-value sentiment task, particularly with P1 setting, achieving a Macro F1 Score of 0.233. The TFIDF component amplifies the distinct sentiment signals by weighing the terms

according to their importance, which is crucial in the nuanced 5-value sentiment scale where the differentiation between adjacent sentiment classes can be subtle. [2]

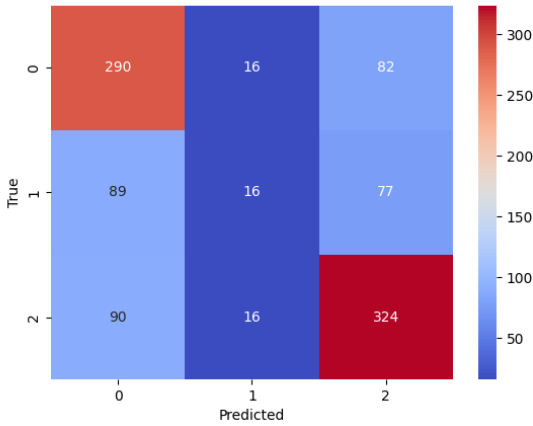
Empirical evidence from the exhaustive feature exploration (as detailed in the Table 1) supports the effectiveness of the selected feature sets. The benefits of these feature sets lie in their complementary strengths—VADER’s nuanced understanding of polarity and TFIDF’s ability to emphasize terms that are critical for sentiment differentiation. However, limitations arise from the complexity of TFIDF’s calculations and VADER’s potential insensitivity to context beyond polarity. In conclusion,

Feature Set (Preprocessing)	5 VALUE	3 VALUE
All Words (NP)	0.238	0.376
All Words (P1)	0.149	0.318
All Words (P2)	0.320	0.509
Adjectives (P2)	0.175	0.300
Negation (P2)	0.153	0.294
Sentiment Shifter (P2)	0.155	0.296
VADER (NP)	0.231	0.403
VADER (P1)	0.229	0.416
VADER (P2)	0.201	0.371
VADER + TFIDF (P1)	0.233	0.387
VADER + TFIDF (P2)	0.195	0.381
VADER + TFIDF + SMOTE (P1/P2)	–couldn’t compute under 3 minutes–	

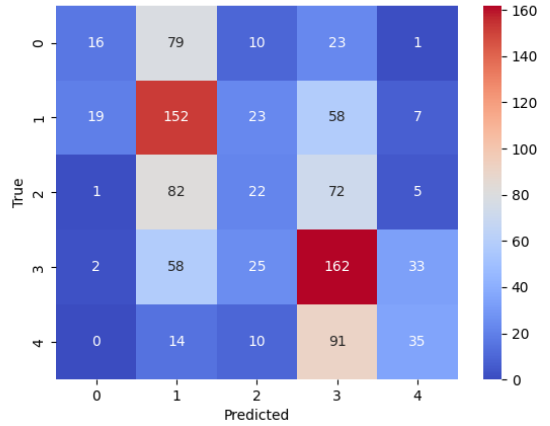
Table 1: Macro F1 Scores for Different Feature Sets with and without Preprocessing

the feature selection for our sentiment analysis task is a product of deliberate empirical exploration, informed by literature and aimed at maximizing the Macro F1 Score. VADER excels in the broader 3-value sentiment classification by capturing the inherent polarity of language, while the VADER + TFIDF combination is superior in the more granular 5-value task by leveraging both polarity and term significance.

3 Result and Discussion



(a) Best Model Confusion Matrix for 3-Value



(b) Best Model Confusion Matrix for 5-Value

Figure 1: Confusion Matrices for Best Models

The Best Model: All Words (P2 Preprocessing)

The empirical evidence, as encapsulated in Table 1 and visualized through Figure 1, clearly illustrates the superior performance of the "All Words" feature set under the P2 preprocessing setting. This model outperforms others with macro-F1 scores of 0.320 for the 5-Value and an impressive 0.509 for the 3-Value Sentiment Scales, thus establishing it as the best model amongst the evaluated candidates.

The P2 setting for "All Words" features is distinguished by its comprehensive coverage of the lexical space, which is pivotal for capturing the varied nuances in movie reviews. This approach ensures that the diversity of the feature space is retained, which is crucial for the following reasons:

- **Lexical Richness:** The richness of expressions used in movie reviews is better captured when the full range of words is considered. Each word can contribute to the overall sentiment, and this is particularly useful in a domain where reviewers use a wide array of vocabulary to express opinions.
- **Context Preservation:** In sentiment analysis, the context in which words are used can significantly alter their sentiment. By maintaining all words, the model benefits from the syntactical and contextual cues inherent in the data, which could be lost with more aggressive feature selection.
- **Feature Synergy:** The interaction between words in a sentence can often convey a sentiment that might not be apparent when considering words in isolation. The "All Words" model inherently captures these interactions, which can be especially informative for sentiment analysis.

While preprocessing is generally a beneficial step in text analytics, there is a caveat in the context of sentiment analysis. Overprocessing can strip away the sentiment-laden context of words, leading to a loss of valuable information. This is evident from the performance dip when feature selection methods like VADER and TFIDF are introduced. These methods, while powerful, rely on a distilled feature set that may neglect subtle sentiment indicators present in the full text, leading to a decline in performance as measured by macro-F1 scores.

In conclusion, the "All Words (P2)" model emerges as the most effective approach for this sentiment analysis task, balancing the need for a comprehensive feature set with the nuances of NLP. The model's efficacy, despite the strong independence assumptions of Naive Bayes as well as class imbalance in the training data, speaks to the richness of the Rotten Tomatoes dataset and the potential for even simple models to achieve high performance when paired with a suitable feature set.

4 Error Analysis

Sentence ID	True Label	Predicted Label	Reason of failure	How to improve the model?
#2663	0	4	Negative Modifiers and Subtle Sarcasm	word embedding
#6586	4	1	No context information and Irony Detection ("amusing")	providing domain specific context and implementing n-grams
#8043	4	1	Interpretation of double negation('no') more strongly than positive('fascinating') and Neutral Descriptors	handling of negations and double negations
#4338	1	4	Implicit sentiment in descriptors ("sophisticated" and "sleazy moralizing") and doesn't understand overall sentence sentiment context	implementing weighting key sentiment indicators

Table 2: Error Analysis for 5-Value Setting

Sentence ID	True Label	Predicted Label	Reason of failure	How to improve the model?
#6885	0	2	Subtlety and Ambiguity of the Phrase	Ensuring the training data includes a wide range of sentences with subtle, mixed or ambiguous sentiments or oversampling using SMOTE.
#540	0	2	Unknown cultural and contextual sensitivities	Using sophisticated models such as LSTM or BERT will help in capturing such sensitivities.
#7414	0	2	Multiple use of clauses in the phrase ("Wannabe comedy of manners," "brainy prep-school kid," and "Mrs. Robinson complex") and a mix of positive and negative connotations	Using n-gram to handle clauses along with VADER/SentiWORDNET to understand phrase's overall sentimental context.
#3177	2	0	Absence of strong sentiment words	Employ techniques that can infer sentiment from minimal information.

Table 3: Error Analysis for 3-Value Setting

References

- [1] C. Hutto and E. Gilbert, “Vader: A parsimonious rule-based model for sentiment analysis of social media text,” *Proceedings of the International AAAI Conference on Web and Social Media*, vol. 8, pp. 216–225, May 2014.
- [2] M. Chiny, M. Chihab, O. Bencharef, and Y. Chihab, “Lstm, vader and tf-idf based hybrid sentiment analysis model,” *International Journal of Advanced Computer Science and Applications*, vol. 12, no. 7, 2021.